Week 3 Lesson 2 — Question 3 Naive Bayes

3rd November 2026

1 Question 3

Let's try one final activity using a fictional set of covid observations and an imaginary patient. Please keep in mind this activity is for practising Naive Bayes calculations only. It should not be used to determine instances of COVID-19 in the real world.

Part 1: Given the table of observations below, predict the output or probability of each, for example, P (chills = Y | covid = Y) =

Part 2: If a patient has the following symptoms: chills, sore throat and a mild headache, are they more likely to have or not have COVID-19?

Chills	Runny nose	Sore throat	Headache	Fever	Covid
Y	N	Y	Mild	Y	N
Y	Y	Y	No	N	Y
Y	N	N	Strong	Y	Y
N	Y	Y	Mild	Y	Y
N	N	N	No	Y	Y
N	Y	Y	Strong	N	N
N	Y	Y	Strong	N	N
Y	Y	N	Mild	Y	Y

[1]:
```
import pandas
import pandas as pd
from IPython.display import HTML
```

```
def style_latex_table(styler):
    """Style LaTeX tables; requires colortbl in the PDF template."""
    def stripe_rows(values):
        return [
            # Wrap the content directly to avoid a leading space after␣
 ↪cellcolor.
            "cellcolor:{gray!10}--rwrap;" if i % 2 == 0 else ""
            for i in range(len(values))
        ]

    def style_index(values):
        return [style + "bfseries:;" for style in stripe_rows(values)]

    return (
        styler
        .apply(stripe_rows, axis=0)
        .apply_index(style_index, axis=0)
        .map_index(lambda value: "bfseries:;", axis=1)
    )

covid_df = pd.DataFrame([
    ("Y", "N", "Y", "Mild", "Y", "N"),
    ("Y", "Y", "Y", "No", "N", "Y"),
    ("Y", "N", "N", "Strong", "Y", "Y"),
    ("N", "Y", "Y", "Mild", "Y", "Y"),
    ("N", "N", "N", "No", "Y", "Y"),
    ("N", "Y", "Y", "Strong", "N", "N"),
    ("N", "Y", "Y", "Strong", "N", "N"),
    ("Y", "Y", "N", "Mild", "Y", "Y"),
], columns=["Chills", "Runny nose", "Sore throat", "Headache", "Fever",␣
 ↪"Covid"])

covid_df
```

[1]:

	Chills	Runny nose	Sore throat	Headache	Fever	Covid
0	Y	N	Y	Mild	Y	N
1	Y	Y	Y	No	N	Y
2	Y	N	N	Strong	Y	Y
3	N	Y	Y	Mild	Y	Y
4	N	N	N	No	Y	Y
5	N	Y	Y	Strong	N	N
6	N	Y	Y	Strong	N	N
7	Y	Y	N	Mild	Y	Y

[2]:
```
print(f"The dataframe has {covid_df.shape[0]} rows and {covid_df.shape[1]}␣
 ↪columns.")
```

```
The dataframe has 8 rows and 6 columns.
```

1.1 Part 1: probability of each

[3]:
```
for column in covid_df.columns:
    print(f"\n{column}")
    counts = covid_df[column].value_counts()

    summary = pd.DataFrame({
        "Fraction": counts.map(lambda count: f"{count}/{len(covid_df)}"),
        "Percentage": counts / len(covid_df),
    }).rename_axis(None).sort_index(ascending=False) # Sort to

    #display(summary.style.format({"Percentage": "{:.2%}"})) # This doesn't␣
 ↪work with nbconvert
    display({ # Double output for the notebook renderer and nbconvert
        "text/html": summary.style.format(
            {"Percentage": "{:.2%}"}
        ).to_html(),
        "text/latex": style_latex_table(summary.style.format(
            {"Percentage": lambda x: f"{x:.2%}".replace("%", r"\%")}
        )).to_latex(),
    }, raw=True)
    #display(summary.assign(Percentage=summary["Percentage"].map("{:.2%}".
 ↪format)))
```

```
Chills
```

	Fraction	Percentage
Y	4/8	50.00%
N	4/8	50.00%

```
Runny nose
```

	Fraction	Percentage
Y	5/8	62.50%
N	3/8	37.50%

```
Sore throat
```

	Fraction	Percentage
Y	5/8	62.50%
N	3/8	37.50%

```
Headache
```

	Fraction	Percentage
Strong	3/8	37.50%
No	2/8	25.00%
Mild	3/8	37.50%

Fever

	Fraction	Percentage
Y	5/8	62.50%
N	3/8	37.50%

Covid

	Fraction	Percentage
Y	5/8	62.50%
N	3/8	37.50%

[4]:
```
covid_yes = covid_df[covid_df["Covid"] == "Y"]

display(covid_yes)

def display_subset_df(subset_df : pd.DataFrame):
    has_covid = subset_df["Covid"].iloc[0]
    has_covid = "Yes" if has_covid == 'Y' else "No"

    for column in subset_df.columns.drop("Covid"):
        print(f"\n{column} given Covid = {has_covid}")
        counts = subset_df[column].value_counts()

        order = ["No", "Mild", "Strong"] if column == "Headache" else ["Y", "N"]
        counts = subset_df[column].value_counts().reindex(order, fill_value=0)

        summary = pd.DataFrame({
            "Fraction": counts.map(lambda count: f"{count}/{len(subset_df)}"),
            "Percentage": counts / len(subset_df),
        }).rename(index={"Y": "Yes", "N": "No"}).rename_axis(None)

        # display(summary.style.format({"Percentage": "{:.2%}"})) # This␣
↪doesn't work with nbconvert
        #display(summary.assign(Percentage=summary["Percentage"].map("{:.2%}".
↪format)))
        display({
            "text/html": summary.style.format(
                {"Percentage": "{:.2%}"}
            ).to_html(),
            "text/latex": style_latex_table(summary.style.format(
                {"Percentage": lambda x: f"{x:.2%}".replace("%", r"\%")}
```

```
            )).to_latex(),
        }, raw=True)

display_subset_df(covid_yes)
```

```
  Chills Runny nose Sore throat Headache Fever Covid
1      Y          Y           Y       No     N     Y
2      Y          N           N   Strong     Y     Y
3      N          Y           Y     Mild     Y     Y
4      N          N           N       No     Y     Y
7      Y          Y           N     Mild     Y     Y
```

Chills given Covid = Yes

	Fraction	Percentage
Yes	3/5	60.00%
No	2/5	40.00%

Runny nose given Covid = Yes

	Fraction	Percentage
Yes	3/5	60.00%
No	2/5	40.00%

Sore throat given Covid = Yes

	Fraction	Percentage
Yes	2/5	40.00%
No	3/5	60.00%

Headache given Covid = Yes

	Fraction	Percentage
No	2/5	40.00%
Mild	2/5	40.00%
Strong	1/5	20.00%

Fever given Covid = Yes

	Fraction	Percentage
Yes	4/5	80.00%
No	1/5	20.00%

[5]:
```
covid_no = covid_df[covid_df["Covid"] == "N"]

display(covid_no)
```

```
display_subset_df(covid_no)
```

```
  Chills Runny nose Sore throat Headache Fever Covid
0      Y          N           Y     Mild     Y     N
5      N          Y           Y   Strong     N     N
6      N          Y           Y   Strong     N     N
```

Chills given Covid = No

	Fraction	Percentage
Yes	1/3	33.33%
No	2/3	66.67%

Runny nose given Covid = No

	Fraction	Percentage
Yes	2/3	66.67%
No	1/3	33.33%

Sore throat given Covid = No

	Fraction	Percentage
Yes	3/3	100.00%
No	0/3	0.00%

Headache given Covid = No

	Fraction	Percentage
No	0/3	0.00%
Mild	1/3	33.33%
Strong	2/3	66.67%

Fever given Covid = No

	Fraction	Percentage
Yes	1/3	33.33%
No	2/3	66.67%

1.2 Part 2: Predicting Covid Using Naive Bayes

[6]:
```
import math

display_labels = {"Y": "Yes", "N": "No"}

def obtain_probability_table(observation: dict[str, str], alpha=0):
    """Build a table of class priors and categorical feature likelihoods.
```

```
    Parameters
    ----------
    observation : dict[str, str]
        Feature names and their observed values, using the dataset's labels:
        "Y"/"N" for binary features; "No", "Mild", or "Strong" for Headache.
        Only supplied features contribute likelihood rows.
    alpha : nonnegative number, default 0
        Additive smoothing strength (a pseudocount per possible feature value).
        For feature F, observed value v, and class c, the likelihood is:

            P(F=v | C=c) = (matching_count + alpha) / (class_count + alpha*K)

        matching_count counts rows with F=v within class c. class_count counts
        all rows in class c. K is the number of distinct values of F across the
        full covid_df, not just within class c: 2 for the binary features and
        3 for Headache.

        alpha=0: no smoothing; use observed frequencies. A zero matching count
        gives a zero likelihood and therefore a zero product score for that
        class. If all class scores are zero, normalization is undefined.

        alpha=1: Laplace (add-one) smoothing. Pretend each possible feature
        value has one additional occurrence in each class. Adding K to the
        denominator keeps the probabilities across that feature's values
        summing to 1 within each class.

        Other positive values give additive smoothing (e.g. alpha=0.5).
        Larger alpha pulls feature probabilities more strongly toward 1/K.
        The intended domain is finite alpha >= 0; this function does not
        currently validate alpha.

        Example: no headache occurs in 0 of the 3 Covid="N" rows. Headache
        has K=3 possible values, so:
            alpha=0: (0 + 0) / (3 + 0*3) = 0/3
            alpha=1: (0 + 1) / (3 + 1*3) = 1/6

        Smoothing applies only to feature likelihoods here. Class priors stay
        at their observed frequencies: P(C=N)=3/8 and P(C=Y)=5/8.
        It does not make invalid labels valid: Headache="N" still raises an
        error; use Headache="No".

    Returns
    -------
    pandas.DataFrame
        Fraction strings with priors and likelihoods as rows and "No"/"Yes"
        class columns. These are P(feature value | class), not posterior
        probabilities P(class | observation). obtain_prediction multiplies
```

```
    the prior and likelihoods, then normalizes the resulting class scores.

Notes
-----
The current obtain_prediction parses fractions with int(), so use integer
alpha values (0, 1, etc.) with that function. To use alpha=0.5 or 1.0,
change its numerator and denominator parsing from int() to float().
K is inferred from observed data; categories absent from the entire
dataset would require an explicitly defined category set.

Examples
--------
obtain_probability_table({"Chills": "Y", "Headache": "Mild"}, alpha=0)
obtain_probability_table({"Chills": "Y", "Headache": "Mild"}, alpha=1)
"""
alpha = float(alpha)

if not math.isfinite(alpha) or alpha < 0:
    raise ValueError("alpha must be a finite, nonnegative number.")

column_to_predict = "Covid"
covid_counts = covid_df[column_to_predict].value_counts()
categories = ["N", "Y"]

# Keep the observed class priors.
fractions_str = {
    f"P({column_to_predict[:1]})": [
        f"{covid_counts[c]}/{len(covid_df)}"
        for c in categories
    ]
}

for feature, value in observation.items():
    allowed_values = covid_df[feature].unique()

    if value not in allowed_values:
        raise ValueError(
            f"{feature}={value!r} is invalid. "
            f"Expected one of {allowed_values.tolist()}."
        )

    # Count matching rows within each Covid category
    matching_counts = (
        covid_df[feature].eq(value)
        .groupby(covid_df[column_to_predict])
        .sum()
    )
```

```
        # .get(value, value) translates Y and N while keeping values such as␣
↪"Mild" unchanged.
        display_value = display_labels.get(value, value)

        k = len(allowed_values)
        if alpha > 0: # Use float values
            fractions_str[f"P({feature}={display_value} | {column_to_predict[:
↪1]})"] = [
                f"{matching_counts[c] + alpha}/{covid_counts[c] + alpha * k}"
                for c in categories
            ]
        else:
            fractions_str[f"P({feature}={display_value} | {column_to_predict[:
↪1]})"] = [
                f"{matching_counts[c]}/{covid_counts[c]}"
                for c in categories
            ]

    return (
        pd.DataFrame(fractions_str, index=categories)
        .T
        .rename(columns=display_labels)
    )

def obtain_prediction(probability_table : pd.DataFrame, debug_mode=False):
    # I will convert the fractions to floats:
    '''probability_table_float = probability_table.map(
        lambda fraction: int(fraction.split("/")[0]) / int(fraction.split("/
↪")[1])
    )'''
    probability_table_float = probability_table.map(
        lambda fraction: (
            float(fraction.split("/")[0])
            / float(fraction.split("/")[1])
        )
    )

    if debug_mode:
        print("Probability table in float")
        display(probability_table_float)

    # I will multiply by column to get the No Covid and Covid probabilities:
    scores = probability_table_float.prod(axis=0)
    if debug_mode:
        print("No Covid and Covid")
        display(scores.to_frame(name="Score"))
```

```
    # Then normalize the scores to get the predicted probabilities:
    total = scores.sum()

    if total == 0:
        raise ValueError(
            "Both class scores are zero. Check observation labels "
            "and zero probabilities before normalizing."
        )
    posterior = scores / total
    if debug_mode:
        print("\nNo Covid and Covid normalised")
        posterior_table = posterior.to_frame(name="Probability")
        display({
            "text/html": posterior_table.style.format("{:.2%}").to_html(),
            "text/latex": style_latex_table(posterior_table.style.format(
                lambda x: f"{x:.2%}".replace("%", r"\%")
            )).to_latex(),
        }, raw=True)

    prediction = posterior.idxmax()  # e..g "No Covid"
    if debug_mode:
        print("\nPrediction: are they more likely to have Covid?", prediction)

    return prediction

# Feature values for the observation we want to predict
# From columns columns=["Chills", "Runny nose", "Sore throat", "Headache",␣
 ↪"Fever", "Covid"])
observation = {
    "Chills": "Y",
    "Sore throat": "Y",
    "Headache": "Mild",
}

probability_table = obtain_probability_table(observation)

display(probability_table)
```

	No	Yes
P(C)	3/8	5/8
P(Chills=Yes | C)	1/3	3/5
P(Sore throat=Yes | C)	3/3	2/5
P(Headache=Mild | C)	1/3	2/5

[7]:
```
prediction = obtain_prediction(probability_table, debug_mode=True)
```

```
Probability table in float
```

```
                             No    Yes
P(C)                   0.375000  0.625
P(Chills=Yes | C)      0.333333  0.600
P(Sore throat=Yes | C) 1.000000  0.400
P(Headache=Mild | C)   0.333333  0.400
```

```
No Covid and Covid
```

```
        Score
No   0.041667
Yes  0.060000
```

```
No Covid and Covid normalised
```

	Probability
No	40.98%
Yes	59.02%

```
Prediction: are they more likely to have Covid? Yes
```

The question asked in part 2 does not match what the solution is giving which is:

- Chills: No
- Runny nose: Yes
- Fever: Yes
- Sore throat: Yes
- Headache: No and Yes (which doesn't exist, must be either Mild of Strong)

[8]:
```
observation = {
    "Chills": "N",
    "Runny nose": "Y",
    "Fever": "Y",
    "Sore throat": "Y",
    "Headache": "No",
}

alpha_parameter = 0

probability_table = obtain_probability_table(observation, alpha=alpha_parameter)

display(probability_table)

prediction = obtain_prediction(probability_table, debug_mode=True)
```

```
                        No  Yes
P(C)                   3/8  5/8
P(Chills=No | C)       2/3  2/5
P(Runny nose=Yes | C)  2/3  3/5
P(Fever=Yes | C)       1/3  4/5
```

```
P(Sore throat=Yes | C)  3/3  2/5
P(Headache=No | C)      0/3  2/5
```

```
Probability table in float
```

```
                             No    Yes
P(C)                   0.375000  0.625
P(Chills=No | C)       0.666667  0.400
P(Runny nose=Yes | C)  0.666667  0.600
P(Fever=Yes | C)       0.333333  0.800
P(Sore throat=Yes | C)  1.000000  0.400
P(Headache=No | C)      0.000000  0.400
```

```
No Covid and Covid
```

```
      Score
No   0.0000
Yes  0.0192
```

```
No Covid and Covid normalised
```

	Probability
No	0.00%
Yes	100.00%

```
Prediction: are they more likely to have Covid? Yes
```

[9]:
```
observation['Headache'] = 'Mild'

probability_table = obtain_probability_table(observation, alpha=alpha_parameter)

#display(probability_table)

prediction = obtain_prediction(probability_table, debug_mode=False)
print(f"Predicion result with Headache: {observation['Headache']}  and alpha =␣
 ↪{alpha_parameter} -> More likely to have Covid: {prediction}")

observation['Headache'] = 'Strong'

probability_table = obtain_probability_table(observation, alpha=alpha_parameter)

#display(probability_table)

prediction = obtain_prediction(probability_table, debug_mode=True)
print(f"Predicion result with Headache: {observation['Headache']}  and alpha =␣
 ↪{alpha_parameter} -> More likely to have Covid: {prediction}")
```

```
Predicion result with Headache: Mild  and alpha = 0 -> More likely to have
Covid: Yes
```

Probability table in float

	No	Yes
P(C)	0.375000	0.625
P(Chills=No | C)	0.666667	0.400
P(Runny nose=Yes | C)	0.666667	0.600
P(Fever=Yes | C)	0.333333	0.800
P(Sore throat=Yes | C)	1.000000	0.400
P(Headache=Strong | C)	0.666667	0.200

No Covid and Covid

	Score
No	0.037037
Yes	0.009600

No Covid and Covid normalised

	Probability
No	79.42%
Yes	20.58%

Prediction: are they more likely to have Covid? No
Predicion result with Headache: Strong and alpha = 0 -> More likely to have Covid: No